

Luz Violeta Macramê

Documentação técnica do desenvolvimento

VERSÃO ATUALIZADA

Banco de imagens: Cloudinary

Vendas e pagamentos: PagBank

Software e documentação técnica desenvolvidos por:

Daniel de Souza Cordeiro

Documento original: 12/02/2026

Revisão vigente: 13/08/2026

Sumário

- Introdução
- Diagrama
- Banco de Dados e Banco de Imagens
- Desenvolvimento Back-end - Arquitetura da API
- Banco de Imagens - Cloudinary
- Vendas e Pagamentos - PagBank
- Desenvolvimento Front-end e Experiência do Usuário
- Otimização, Performance e Disponibilidade Mobile

Nota de revisão: as decisões Firebase Storage e EFI Bank descritas na versão de 12/02/2026 foram substituídas, respectivamente, por Cloudinary e PagBank.

Introdução

Com o crescimento de sua marca e a expansão do alcance de seu trabalho, a artesã Camila Ribeiro Pedroso decidiu levar suas criações - que unem o mundo real ao espiritual - para o universo digital.

A Luz Violeta Macramê, nascida em Itu/SP, já percorreu diversas regiões do Sul e Sudeste do Brasil em busca de conhecimento, conexões e da oportunidade de compartilhar suas energias através da arte. Agora, dá mais um passo importante em sua jornada: marcar presença no ambiente digital.

O site luz-violeta-macrame.com.br foi concebido para múltiplas plataformas, com experiência responsiva para smartphones, desktops, notebooks e tablets.

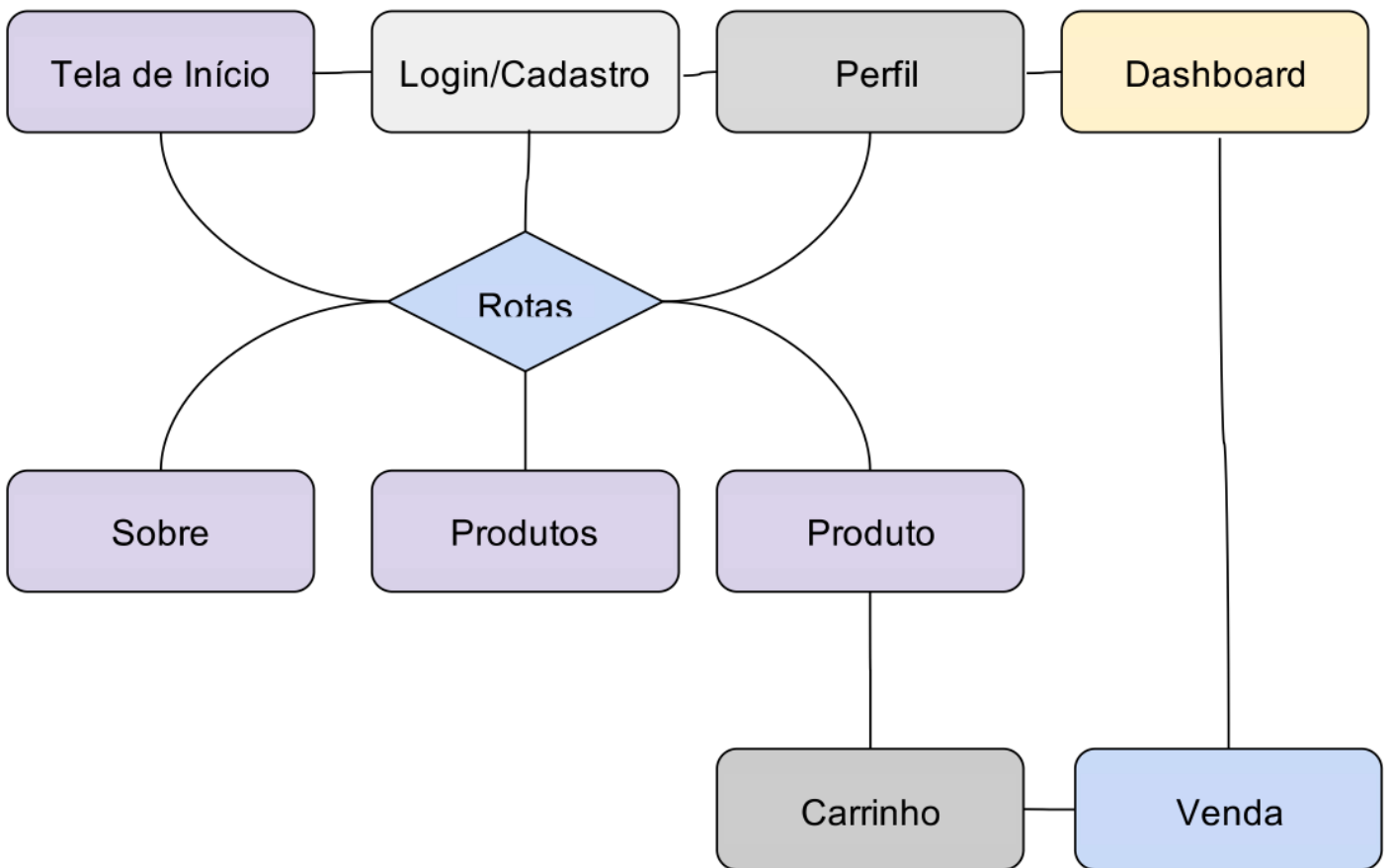
A aplicação web utiliza tecnologias modernas no front-end e integra-se a uma API Node.js. As decisões vigentes para persistência de dados, banco de imagens e pagamentos são:

Back-end: Express.js, JavaScript e MongoDB; integração de imagens com Cloudinary

Front-end: Angular 20, TypeScript, Sass, Material Design e RxJS

Serviço de venda e pagamento: PagBank, integrado exclusivamente pelo back-end

Diagrama



Banco de Dados e Banco de Imagens

Para atender às necessidades da aplicação desenvolvida em Node.js, foi adotado o MongoDB Atlas como banco de dados. O plano e o dimensionamento devem ser reavaliados conforme o volume real de requisições, produtos, clientes e pedidos.

O MongoDB armazena documentos JSON e integra-se à arquitetura JavaScript da API. Ele permanece como fonte dos dados de negócio: usuários, produtos, estoque, carrinhos, pedidos, vendas e referências dos arquivos de imagem.

Imagens e outros binários não devem ser gravados diretamente nos documentos do banco. Para essa finalidade, a decisão vigente é o Cloudinary, em substituição ao Google Firebase Storage.

Após cada upload, a API deve persistir no MongoDB apenas os metadados necessários, como `secure_url`, `public_id`, `asset_id`, `formato`, `dimensões` e `vínculo` com o produto. Esses identificadores também permitem substituir ou excluir o ativo sem depender somente da URL pública.

A configuração definitiva de índices, validações, limites e política de retenção precisa ser confirmada no repositório do back-end antes da publicação.

Desenvolvimento Back-end – Arquitetura da API

A API foi desenvolvida em **JavaScript**, utilizando o framework **Express.js**, com padronização de código configurada via **ESLint (Airbnb Style Guide)**.

Sua arquitetura foi projetada para garantir organização, escalabilidade e fácil manutenção, adotando um fluxo baseado em **Controllers**, que direcionam requisições para diferentes páginas e manipulam dados de forma isolada.

A maior parte da API segue um esquema completo de **CRUD**, estruturado nos seguintes módulos:

- **Schemas (Models)**
- **Controllers**
- **Services**
- **Middlewares**
- **Config**

Persistência e Validação de Dados

A conexão com o banco de dados é realizada via **Mongoose**, integrando a API ao **MongoDB**.

Antes de criar ou atualizar qualquer registro, é aplicada uma validação utilizando **Yup**, garantindo que todos os dados enviados pelo Front-end estejam corretos e sigam o padrão esperado.

Autenticação e Segurança

O sistema de autenticação utiliza **JWT (JSON Web Token)**, assegurando acesso restrito às rotas protegidas.

Todas as senhas são armazenadas com segurança, através de **hashing com Bcrypt**, reforçando a proteção dos dados dos usuários.

Banco de Imagens - Cloudinary

O Cloudinary é o serviço definido para upload, armazenamento, transformação, otimização e entrega das imagens do catálogo. Ele substitui a decisão histórica de uso do Firebase Storage.

Fluxo recomendado

- O administrador autenticado seleciona a imagem no front-end.
- O back-end valida autorização, tipo MIME, tamanho e quantidade de arquivos.
- O upload ocorre no back-end ou diretamente no navegador com assinatura curta gerada pelo servidor.
- A API salva no MongoDB a URL HTTPS e os identificadores `public_id/asset_id` retornados pelo Cloudinary.
- Substituição e exclusão usam o identificador do ativo; credenciais nunca são expostas no Angular.
- Transformações de tamanho, formato e qualidade devem ser padronizadas para catálogo, detalhe e miniaturas.

Estado em 13/08/2026: Cloudinary é a decisão vigente. O contrato e a implementação efetiva precisam ser confirmados no back-end antes de considerar a migração concluída.

Vendas e Pagamentos - PagBank

O PagBank é o provedor definido para pedidos e pagamentos, em substituição ao EFL Bank. A integração deve ser executada pelo back-end; token, credenciais e chaves não podem ser enviados ao navegador.

Fluxo obrigatório da venda

- O back-end recebe apenas IDs e quantidades, revalida estoque e recalcula itens, frete e total.
- Um pedido local nasce como pendente, com referência única e chave de idempotência.
- O servidor cria o pedido no PagBank e devolve ao front-end somente os dados necessários, como QR Code PIX ou link de checkout.
- A venda só muda para paga após webhook autenticado ou consulta server-side confirmando o status do PagBank.
- Reenvios de webhook devem ser idempotentes; cancelamento, expiração, recusa e estorno precisam atualizar o pedido local.

Estados mínimos

Pendente, aguardando pagamento, em análise (quando aplicável), pago, recusado, cancelado, expirado e estornado. Os nomes internos devem ser mapeados para os status oficiais do método utilizado.

Estado em 13/08/2026: o front-end ainda contém fluxo PIX legado e marca a venda como concluída antes da confirmação. Portanto, o pagamento não está pronto para produção.

Desenvolvimento Front-end e Experiência do Usuário

Todas as telas da aplicação foram projetadas para criar uma harmonia entre a estética artesanal da cliente e a identidade visual moderna proporcionada pelo **Angular** aliado ao **Material Design**. O objetivo foi unir sensibilidade artística e tecnologia, entregando uma experiência visual agradável, intuitiva e coerente.

A padronização de cores e estilos foi desenvolvida utilizando **Sass (CSS3)**, que também foi empregado para criar **mixins de responsividade**, permitindo adaptar a interface a diferentes tamanhos de tela — **desktop, notebook, tablets e smartphones** — mantendo ações consistentes mesmo em layouts distintos.

Formulários e Usabilidade

Grande parte das telas administrativas (cerca de 80%) envolve formulários. Para garantir um preenchimento seguro e confiável, foi utilizado o **Angular Forms**, que reduz significativamente a ocorrência de erros.

Entre os recursos adotados:

- Validações robustas
- Mensagens claras de alerta
- Bloqueio do envio em caso de dados inválidos

Essas medidas elevam a qualidade da experiência da administradora e reduzem erros operacionais.

Facilidade de Navegação

Para os usuários finais, a aplicação prioriza clareza e simplicidade.

Elementos como:

- **Toasts informativos**
- **Interfaces minimalistas**

ajudam a guiar o usuário durante toda a navegação, tornando a experiência leve e acessível.

Otimização e Performance

Diversas telas utilizam o **localStorage** para armazenar dados previamente carregados da API. O site exibe mais fluido e eficiente, especialmente em dispositivos móveis. Esse recurso reduz a necessidade de consultas repetidas ao servidor e traz benefícios como:

- Menor número de requisições ao Back-end
- Economia de processamento e tráfego
- Maior velocidade no carregamento das telas

Disponibilidade em Dispositivos Mobile

A aplicação foi projetada com foco em responsividade e adaptação completa ao ambiente mobile. Isso permite oferecer ao usuário a opção de:

- Instalar o app em seu dispositivo
- Navegar com uma interface totalmente otimizada para telas menores

O resultado é uma experiência consistente e funcional em qualquer plataforma.